

TRABAJO PRÁCTICO: Lombok, DTOs y Servicios Web REST con Spring Boot

Objetivo

1. Refactorizar y simplificar el modelo de entidades JPA eliminando código repetitivo mediante **Lombok**.
2. Optimizar el rendimiento de las consultas y proteger la capa de dominio implementando **DTOs** a través de construcciones complejas en **JPQL**.
3. Exponer un servicio web **RESTful** utilizando **Spring Boot** que devuelva información filtrada en formato **JSON**.

Parte 1: Integración de Lombok en el Modelo de Dominio

Consignas

1. Agregar la dependencia de Lombok (org.projectlombok:lombok) en el archivo de construcción de proyecto (pom.xml o build.gradle).
2. Refactorizar las superclases (EntityId, AuditoriaApp) y las entidades concretas (FacturaVenta, Cliente, Articulo, etc.) sustituyendo los métodos manuales (*getters*, *setters*, constructores) por las anotaciones de Lombok.
3. **Cuidado con JPA y Lombok:**
 - Aplicar @Getter y @Setter en lugar de @Data en entidades con relaciones bidireccionales (@OneToMany / @ManyToOne), o bien personalizar @ToString.Exclude y @EqualsAndHashCode.Exclude para **evitar bucles de recursión infinita (StackOverflowError)**.
 - En las clases que heredan de EntityId o AuditoriaApp, incluir @EqualsAndHashCode(callSuper = true).

Ejemplo de Código Refactorizado

```
import lombok.*;
import javax.persistence.*;
import java.util.Date;
import java.util.List;

// Superclase base
@MappedSuperclass
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public abstract class EntityId {
```

```

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    protected Long id;
}

// Entidad FacturaVenta refactorizada con Lombok
@Entity
@Table(name = "factura_venta")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
@Builder
@EqualsAndHashCode(callSuper = true, exclude = {"detalles"})
@ToString(exclude = {"detalles"})
public class FacturaVenta extends AuditoriaApp {

    private Long numero;

    @Column(nullable = false)
    @Temporal(TemporalType.DATE)
    private Date fechaEmision;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "punto_venta_id", nullable = false)
    private PuntoVenta puntoVenta;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "cliente_id", nullable = true)
    private Cliente cliente;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "condicion_iva_id", nullable = false)
    private CondicionIva condicionIva;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "tipo_moneda_id", nullable = false)
    private TipoMoneda tipoMoneda;

    private double importeCobrado;
    private double importeSaldo;

    @Column(nullable = false)
    private double importeTotal;
}

```

```

private String cae;
private Date caeFechaVencimiento;
private String resultadoAfip;
private String motivoRechazo;

@Column(nullable = false)
private String estado;

private Date fechaAnulacion;
private String observaciones;

@OneToMany(mappedBy = "factura", cascade = CascadeType.ALL, orphanRemoval =
true)
private List<FacturaVentaDetalle> detalles;
}

```

Parte 2: Proyecciones DTO con JPQL para Reportes

Consignas

1. Crear una clase **DTO** nombrada FacturaReporteDTO que contenga únicamente los datos resumidos para un reporte ejecutivo de ventas:
 - o Long numeroFactura
 - o Date fechaEmision
 - o String clienteDenominacion (o "Consumidor Final" si es nulo)
 - o String condicionIva
 - o String puntoVentaDescripcion
 - o double importeTotal
 - o long cantidadItems (cantidad de ítems/detalles que posee la factura)
2. Utilizar Lombok (@Getter, @AllArgsConstructor, @NoArgsConstructor) para definir el DTO de forma concisa.
3. Escribir una consulta **JPQL** que haga uso de la expresión de constructor **SELECT new com.ejemplo.dto.FacturaReporteDTO(...)** para proyectar directamente desde la base de datos hacia el DTO sin mapear entidades completas en memoria.

Código del DTO (FacturaReporteDTO.java)

```

package com.ejemplo.dto;

import lombok.AllArgsConstructor;
import lombok.Getter;
import lombok.NoArgsConstructor;
import lombok.Setter;

```

```

import java.util.Date;

@Getter
@Setter
@AllArgsConstructor
@NoArgsConstructor
public class FacturaReporteDTO {
    private Long numeroFactura;
    private Date fechaEmision;
    private String clienteDenominacion;
    private String condicionIva;
    private String puntoVentaDescripcion;
    private double importeTotal;
    private long cantidadItems;
}

```

Consulta JPQL

```

String jpql = "SELECT new com.ejemplo.dto.FacturaReporteDTO(" +
    "    f.numero, " +
    "    f.fechaEmision, " +
    "    COALESCE(c.denominacion, 'Consumidor Final'), " +
    "    ci.denominacion, " +
    "    pv.descripcion, " +
    "    f.importeTotal, " +
    "    COUNT(d) " +
    ") " +
    "FROM FacturaVenta f " +
    "LEFT JOIN f.cliente c " +
    "JOIN f.condicionIva ci " +
    "JOIN f.puntoVenta pv " +
    "JOIN f.detalles d " +
    "GROUP BY f.id, f.numero, f.fechaEmision, c.denominacion, " +
    "ci.denominacion, pv.descripcion, f.importeTotal";

List<FacturaReporteDTO> reporte = em.createQuery(jpql,
FacturaReporteDTO.class).getResultList();

```

Aplique la consulta JPQL para la generación de 2 archivos, uno en formato PDF y el otro como hoja de calculo (excel), investigue y utilice las librerías que desee para generar los archivos en los tipos solicitados por ejemplo para PDF puede usar itext y para el Excel puede generar un txt separado por tabulaciones.

Parte 3: API REST con Spring Boot, DTOs y Filtros Opcionales

Consignas

1. Crear un proyecto **Spring Boot** (versión 3.x o superior) incluyendo las dependencias:
 - Spring Web
 - Spring Data JPA
 - MySQL Driver / PostgreSQL Driver
 - Lombok
2. Crear un componente de repositorio o servicio (FacturaService / FacturaRepository) que implemente la búsqueda dinámica de facturas devolviendo una lista de FacturaReporteDTO.
3. Crear un controlador REST (FacturaRestController) mapeado en el path /api/facturas.
4. Implementar un método HTTP **GET** que retorne una lista de FacturaReporteDTO en formato **JSON**.
5. El endpoint debe aceptar parámetros de consulta opcionales (@RequestParam) como filtros:
 - fechaDesde (opcional): Filtra facturas desde esta fecha.
 - fechaHasta (opcional): Filtra facturas hasta esta fecha.
 - estado (opcional): Filtra por estado (ej. "EMITIDA", "ANULADA").
 - montoMinimo (opcional): Filtra facturas cuyo importeTotal sea mayor o igual al monto especificado.

Ejemplo Método de Consulta

```
public List<FacturaReporteDTO> buscarFacturasFiltradas(Date fechaDesde, Date
fechaHasta, String estado, Double montoMinimo) {
    StringBuilder jpql = new StringBuilder(
        "SELECT new com.ejemplo.dto.FacturaReporteDTO(" +
        "    f.numero, f.fechaEmision, COALESCE(c.denominacion, 'Consumidor
Final'), " +
        "    ci.denominacion, pv.descripcion, f.importeTotal, COUNT(d)" +
        ") " +
        "FROM FacturaVenta f " +
        "LEFT JOIN f.cliente c " +
        "JOIN f.condicionIva ci " +
        "JOIN f.puntoVenta pv " +
        "JOIN f.detalles d " +
        "WHERE 1=1 "
```

```

);

Map<String, Object> params = new HashMap<>();

if (fechaDesde != null) {
    jpql.append("AND f.fechaEmision >= :fechaDesde ");
    params.put("fechaDesde", fechaDesde);
}
if (fechaHasta != null) {
    jpql.append("AND f.fechaEmision <= :fechaHasta ");
    params.put("fechaHasta", fechaHasta);
}
if (estado != null && !estado.trim().isEmpty()) {
    jpql.append("AND f.estado = :estado ");
    params.put("estado", estado);
}
if (montoMinimo != null) {
    jpql.append("AND f.importeTotal >= :montoMinimo ");
    params.put("montoMinimo", montoMinimo);
}

jpql.append("GROUP BY f.id, f.numero, f.fechaEmision, c.denominacion,
ci.denominacion, pv.descripcion, f.importeTotal");

TypedQuery<FacturaReporteDTO> query = em.createQuery(jpql.toString(),
FacturaReporteDTO.class);
params.forEach(query::setParameter);

return query.getResultList();
}

```

Ejemplo de Respuesta JSON Esperada en el Cliente (Postman / Browser)

```

[
  {
    "numeroFactura": 1001,
    "fechaEmision": "2026-03-15",
    "clienteDenominacion": "Consumidor Final",
    "condicionIva": "IVA Responsable Inscripto",
    "puntoVentaDescripcion": "Sucursal Central",
    "importeTotal": 1500.0,
    "cantidadItems": 2
  },
  {
    "numeroFactura": 1002,
    "fechaEmision": "2026-03-20",

```

```
    "clienteDenominacion": "Empresa Tech S.A.",  
    "condicionIva": "IVA Responsable Inscripto",  
    "puntoVentaDescripcion": "Sucursal Central",  
    "importeTotal": 85000.5,  
    "cantidadItems": 5  
  }  
]
```